

System Design Write-Up: Encore Ticket Booking Platform

AJITESH SHARMA
VIT VELLORE BATCH 2023 PASSOUT 2027
COMPUTER SCIENCE AND ENGINEERING WITH SPEC IN INFORMATION SECURITY AS BRANCH
TICKET BOOKING AS TITLE OF THE PROJECT

1. Introduction and Architecture Overview

The Encore Ticket Booking platform is a distributed, highly concurrent system built to solve the notoriously difficult "FairHold" problem—preventing double-bookings when thousands of users attempt to purchase the exact same seat simultaneously. The architecture utilizes a Next.js (React) frontend deployed on Vercel, a NestJS backend API deployed on Render, and a highly normalized PostgreSQL database (Neon) interfaced via Drizzle ORM.

2. Concurrency Prevention (The FairHold Problem)

In a high-demand ticketing scenario, traditional row-level locking or basic SELECT checks lead to race conditions where two users are mistakenly sold the same ticket. Encore mitigates this through a combination of Optimistic Concurrency Control (OCC) and atomic state transitions:

- Strict State Machine:** The `show_seats` table enforces a strict state machine: `available` -> `held` -> `booked`.
- Version Control (OCC):** Every seat has an integer version column. When a hold request is initiated, the database performs an atomic update:
`UPDATE show_seats SET status = 'held', version = version + 1 WHERE id = X AND status = 'available' AND version = Y`. If another user modified the row milliseconds prior, the version increments, the WHERE clause fails, and the database safely aborts the transaction with zero double-bookings.
- Database-Level Locks:** For highly sensitive queue operations (like claiming a seat), we utilize `FOR UPDATE SKIP LOCKED` inside explicit Postgres transactions. This allows multiple concurrent workers to scan for available seats without blocking each other, dramatically increasing throughput.

3. Seat Hold and TTL (Time-To-Live) Mechanism

To prevent malicious actors from hoarding tickets indefinitely, Encore implements a rigid Time-To-Live (TTL) mechanism natively integrated into the database layer.

When a user selects a seat, a 15-minute cryptographic hold is generated. The `show_seats` table is updated with a `heldByUserId` and a strict timestamp `heldUntil`. A corresponding session record is created in the `holds` table. During the checkout process, all subsequent validations—generating payment intents, finalizing bookings, or applying discounts—cross-check the current server timestamp against `heldUntil`. If the server time exceeds the TTL, the API immediately throws an `HTTP 409 Conflict` error, rendering the cart instantly invalid without requiring a background cron job to step in first.

4. Background Worker and Garbage Collection

While the TTL natively prevents expired checkouts, the system must forcefully reclaim these abandoned seats to sell them to other users. Encore utilizes an asynchronous Background Worker pattern (operating via BullMQ with Redis, or falling back to a recursive database poller).

This worker routinely claims asynchronous jobs from the database. When executing a `release_expired_holds` job, it wraps the following in a single ACID transaction: 1. Identifies all rows in `show_seats` where `status = 'held'` and `heldUntil <= now()`. 2. Resets their status back to `available` and clears the user binding. 3. Marks the parent `holds` session as `cancelled`. By batching these operations inside a worker rather than executing them synchronously during user requests, the main API remains highly responsive and decoupled from heavy sweeping operations.

5. Waitlist Auto-Assignment Flow

One of the most complex features of Encore is the automated Waitlist Engine. When a ticket category sells out, users can opt into a waitlist. This operates as a strict **FIFO (First-In, First-Out)** queue natively inside the database.

The waitlist logic is deeply integrated into the garbage collection worker. When the worker forcefully releases an expired hold, it does not immediately release the seat back to the general public. Instead, it triggers the `allocateWaitlist` routine: 1. The worker queries the `waitlist_entries` table

for the oldest user waiting for that specific show and seat category, using `FOR UPDATE SKIP LOCKED` to prevent concurrent workers from claiming the same waitlist entry. 2. The system bypasses the public pool and directly updates the `show_seats` table, assigning a new 15-minute `heldUntil` TTL directly to the waitlisted user. 3. The waitlist entry is updated from `waiting` to `offered`, and given an `offerExpiresAt` timestamp. 4. An event is dispatched to an email service (Resend) to notify the customer that they have a 15-minute window to claim their reserved pass.

6. Time-Limited Offer Handling

If the waitlisted user fails to complete their purchase within their 15-minute window, the exact same garbage collection cycle takes over. The background worker detects that the seat is past its `heldUntil` TTL, and that the waitlist entry is past its `offerExpiresAt` limit. The worker marks the waitlist entry as `expired`, revokes the hold, and automatically allocates the seat to the *next* person in the FIFO queue.

This creates a highly efficient, self-healing loop. The system continuously cascades abandoned tickets down the waitlist without any human intervention from the event organiser, ensuring maximum venue capacity and revenue realization.